Blood Bank Inventory Dashboard

Python source code using Streamlit, Pandas, NumPy, and Matplotlib

```
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from datetime import date

# --------------------------------
# 1. PAGE CONFIGURATION
# --------------------------------

st.set_page_config(
    page_title="Blood Bank Dashboard",
    page_icon="■",
    layout="wide"
)

# --------------------------------
# 2. SAMPLE BLOOD BANK DATA
# --------------------------------

blood_groups = [
    "A+", "A-", "B+", "B-",
    "O+", "O-", "AB+", "AB-"
]

initial_units = [45, 12, 38, 8, 60, 10, 15, 5]

# --------------------------------
# 3. SESSION STATE
# --------------------------------

if "inventory" not in st.session_state:
    st.session_state.inventory = pd.DataFrame({
        "Blood Group": blood_groups,
        "Available Units": initial_units
    })

if "collected" not in st.session_state:
    st.session_state.collected = 85

if "issued" not in st.session_state:
    st.session_state.issued = 61

if "transactions" not in st.session_state:
    st.session_state.transactions = []

# --------------------------------
# 4. PAGE TITLE
# --------------------------------

st.title("■ Blood Bank Inventory Dashboard")

st.write(
    "Blood Inventory Analysis and Monthly Reports"
)

st.divider()

# --------------------------------
# 5. INVENTORY SUMMARY
# --------------------------------

total_available = int(
    st.session_state.inventory["Available Units"].sum()
)

total_collected = st.session_state.collected
total_issued = st.session_state.issued

low_stock = int(
    (st.session_state.inventory["Available Units"] <= 10).sum()
)

st.subheader("■ Inventory Overview")

col1, col2, col3, col4 = st.columns(4)

col1.metric("Available Units", total_available)
col2.metric("Collected Units", total_collected)
col3.metric("Issued Units", total_issued)
col4.metric("Low Stock Groups", low_stock)

st.divider()
```

```
# --------------------------------
# 6. BLOOD GROUP INVENTORY TABLE
# --------------------------------

st.subheader("■ Blood Group Inventory")

inventory = st.session_state.inventory.copy()

def stock_status(units):
    if units <= 10:
        return "Low Stock"
    return "Available"

inventory["Status"] = inventory["Available Units"].apply(
    stock_status
)

st.dataframe(
    inventory,
    hide_index=True,
    width="stretch"
)

# --------------------------------
# 7. ADD BLOOD UNITS
# --------------------------------

st.divider()

st.subheader("■ Add Blood Units")

with st.form("add_blood_form"):

    add_group = st.selectbox(
        "Select Blood Group",
        blood_groups,
        key="add_group"
    )

    add_units = st.number_input(
        "Enter Units to Add",
        min_value=1,
        max_value=1000,
        value=1,
        step=1
    )

    add_button = st.form_submit_button(
        "Add Blood Units"
    )

    if add_button:

        index = st.session_state.inventory[
            st.session_state.inventory["Blood Group"] == add_group
        ].index[0]

        st.session_state.inventory.loc[
            index, "Available Units"
        ] += add_units

        st.session_state.collected += add_units

        st.session_state.transactions.append({
            "Date": date.today(),
            "Blood Group": add_group,
            "Transaction": "Collected",
            "Units": add_units
        })

        st.success(
            f"{add_units} units added successfully to {add_group}!"
        )

# --------------------------------
# 8. ISSUE BLOOD UNITS
# --------------------------------

st.subheader("■ Issue Blood Units")

with st.form("issue_blood_form"):

    issue_group = st.selectbox(
        "Select Blood Group",
        blood_groups,
        key="issue_group"
    )

    issue_units = st.number_input(
        "Enter Units to Issue",
        min_value=1,
```

```
        max_value=1000,
        value=1,
        step=1
    )

    issue_button = st.form_submit_button(
        "Issue Blood Units"
    )

    if issue_button:

        index = st.session_state.inventory[
            st.session_state.inventory["Blood Group"] == issue_group
        ].index[0]

        available = int(
            st.session_state.inventory.loc[
                index, "Available Units"
            ]
        )

        if issue_units <= available:

            st.session_state.inventory.loc[
                index, "Available Units"
            ] -= issue_units

            st.session_state.issued += issue_units

            st.session_state.transactions.append({
                "Date": date.today(),
                "Blood Group": issue_group,
                "Transaction": "Issued",
                "Units": issue_units
            })

            st.success(
                f"{issue_units} units issued successfully from {issue_group}!"
            )

        else:
            st.error(
                f"Not enough stock! Available units: {available}"
            )

# --------------------------------
# 9. BLOOD INVENTORY GRAPH
# --------------------------------

st.divider()

st.subheader("■ Blood Group Inventory Analysis")

fig, ax = plt.subplots(figsize=(10, 5))

ax.bar(
    st.session_state.inventory["Blood Group"],
    st.session_state.inventory["Available Units"]
)

ax.set_xlabel("Blood Groups")
ax.set_ylabel("Available Units")
ax.set_title("Available Blood Units by Group")

ax.grid(axis="y", alpha=0.3)

st.pyplot(fig)
plt.close(fig)

# --------------------------------
# 10. LOW STOCK ALERT
# --------------------------------

st.divider()

st.subheader("■■ Low Stock Alert")

low_inventory = st.session_state.inventory[
    st.session_state.inventory["Available Units"] <= 10
]

if not low_inventory.empty:

    st.warning("Some blood groups have low inventory!")

    st.dataframe(
        low_inventory,
        hide_index=True,
        width="stretch"
    )
else:
```

```
    st.success("All blood groups have sufficient stock.")

# --------------------------------
# 11. MONTHLY REPORT
# --------------------------------

st.divider()

st.subheader("■ Monthly Inventory Report")

if st.session_state.transactions:

    report = pd.DataFrame(
        st.session_state.transactions
    )

    report["Date"] = pd.to_datetime(report["Date"])

    report["Month"] = report["Date"].dt.strftime("%Y-%m")

    monthly_report = report.groupby(
        ["Month", "Transaction"]
    )["Units"].sum().unstack(fill_value=0)

    st.dataframe(
        monthly_report,
        width="stretch"
    )

    st.subheader("Transaction History")

    st.dataframe(
        report.drop(columns=["Month"]),
        hide_index=True,
        width="stretch"
    )

    csv_data = report.to_csv(index=False).encode("utf-8")

    st.download_button(
        label="■ Download Monthly Report",
        data=csv_data,
        file_name="blood_bank_monthly_report.csv",
        mime="text/csv"
    )

else:
    st.info(
        "Monthly transaction report will appear after adding or issuing blood units."
    )

# --------------------------------
# 12. FOOTER
# --------------------------------

st.divider()

st.caption(
    "Blood Bank Inventory Dashboard | Developed using Python and Streamlit"
)

st.caption(
    "Note: This project uses sample data for demonstration."
)
```